Bits, Data Types, and Operations

CSE 100/1000/1100: Introduction to Computer Engineering

Instructor:
Fatma CORUT ERGİN

Slides adapted from

- *McGraw Hill Companies, and*
- *Walid A. Najjar & Brian J. Linard*

slides

How do we represent data in a computer?

At the lowest level, a computer is an electronic machine.

- **works by controlling the flow of electrons**

Easy to recognize two conditions:

1. **presence of a voltage – we'll call this state "1"**
2. **absence of a voltage – we'll call this state "0"**

Could base state on *value* of voltage,
but control and detection circuits more complex.

- **compare turning on a light switch to measuring or regulating voltage**

Computer is a binary digital system.

Digital system:

- finite number of symbols

Binary (base two) system:

- has two states: 0 and 1



Basic unit of information is the *binary digit*, or *bit*.

Values with more than two states require multiple bits.

- **A collection of two bits has four possible states:**
  00, 01, 10, 11
- **A collection of three bits has eight possible states:**
  000, 001, 010, 011, 100, 101, 110, 111
- ***A collection of n bits has 2^n possible states.***

What kinds of data do we need to represent?

- **Numbers – signed, unsigned, integers, floating point, complex, rational, irrational, …**
- **Text – characters, strings, …**
- **Images – pixels, colors, shapes, …**
- **Sound**
- **Logical – true, false**
- **Instructions**
- **…**

Data type:

- ***representation* and *operations* within the computer**

We'll start with numbers…

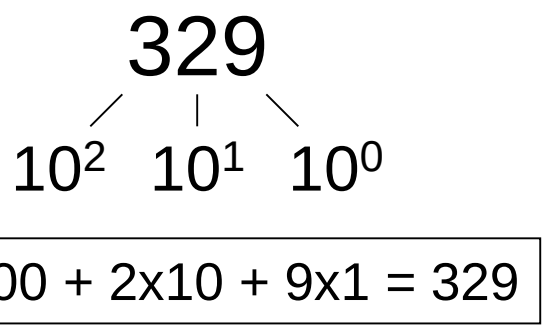
Unsigned Integers (only positive values)

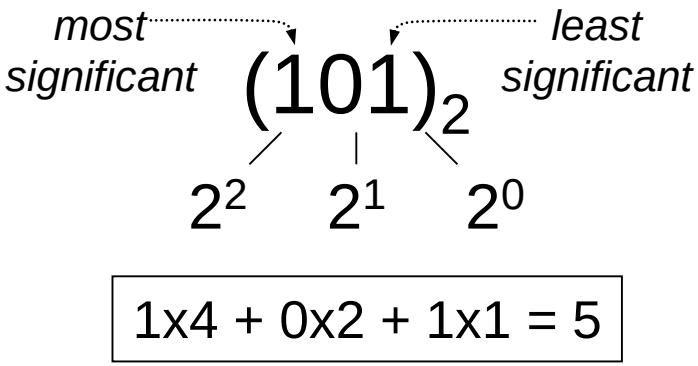
Non-positional notation

- **could represent a number (“5”) with a string of ones (“11111”)**
- **problems?**

Weighted positional notation

- **like decimal numbers: “329”**
- **“3” is worth 300, because of its position, while “9” is only worth 9**





Unsigned Integers (cont.)

An *n*-bit unsigned integer represents 2^n different values: from 0 to 2^n-1.

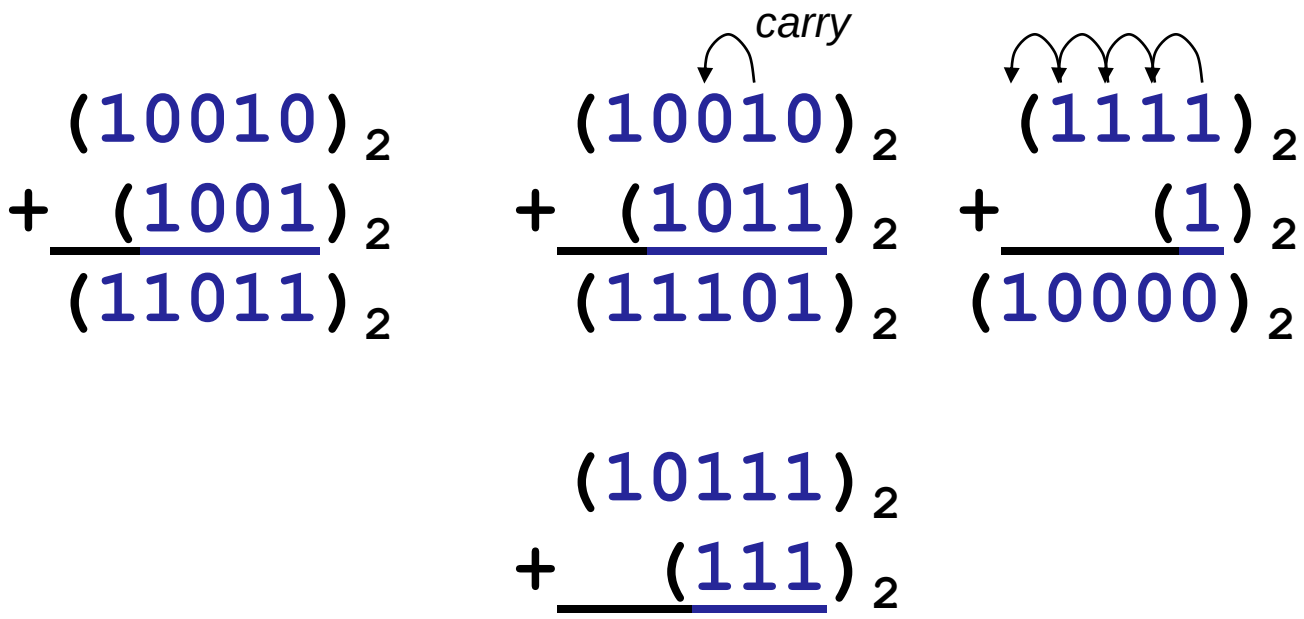
With 3 bits (*n=3*), we can represent the integers from 0 to 7 (2^3-1)

2^2	2^1	2^0	
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

Unsigned Binary Arithmetic

Base-2 addition – just like base-10!

- **add from right to left, propagating carry**



Subtraction, multiplication, division,…

Signed Integers (both positive and negative values)

With n bits, we have 2^n distinct values.

- **assign about half to positive integers (1 through $2^{n-1}-1$) and about half to negative (- ($2^{n-1}-1$) through -1)**

- **that leaves two values: one for 0, and one extra**

  - ***e.g.* with 3 bits we can generate 8 different values.**
  - **half for positives: 1 to 3 and**
  - **half for negatives: -3 to -1 and**
  - **one for 0**
  - **-3, -2, -1, 0, 1, 2, 3**

Signed Integers (cont.)

3 different representations (data type)

1. **signed magnitude**
2. **one's complement**
3. **two's complement**

Positive values (same for all three representations)

- **just like unsigned – '0' in *most significant (MS)* bit**
  $(00101)_2 = 5$

Negative values ('1' in *most significant (MS)* bit)

- **signed magnitude**
  - $(10101)_2 = -5$ $(0101)_2 = 5$
- **one's complement**
  - $(10101)_2 = -10$ flip bits except MS bit $(1010)_2 = 10$
- **two's complement**
  - $(10101)_2 = -11$ see next slides

Problems with signed-magnitude and one's complement

- **two representations of zero (+0 and –0)**
  - **00000 and 10000 in signed-magnitude**
  - **00000 and 11111 in one's complement**

- **arithmetic circuits are complex**
  - **How to add two signed-magnitude numbers?**
    - **e.g., try 2 + (-3) → 00010 + 10011 = 10101**
  - **How to add two one's complement numbers?**
    - **e.g., try 4 + (-3) → 00100 + 11100 = 10000**

Two's Complement

Developed to make circuits easy for arithmetic

- **for each positive number (X), assign value to its negative (-X), such that X + (-X) = 0 with "normal" addition, ignoring carry out**

	00101	(5)
+	11011	(-5)
	00000	(0)

	01001	(9)
+	10111	(-9)
	00000	(0)

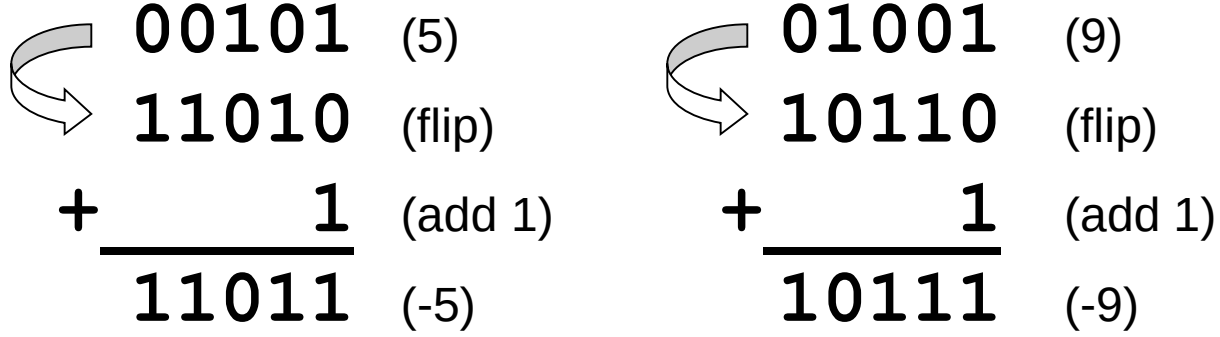
How to Represent a Number using Two's Complement Representation

If number is positive or zero,

- **normal binary representation, zeroes in upper bit(s)**

If number is negative,

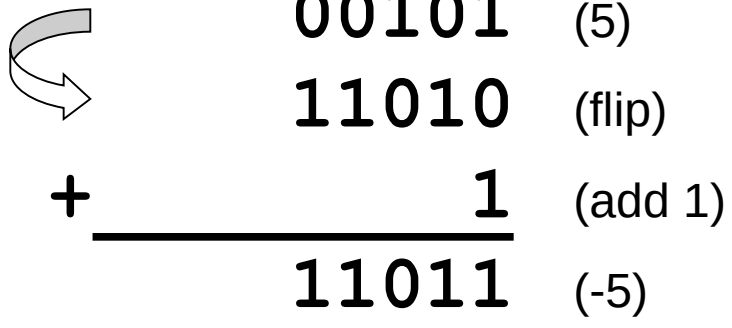
- **start with positive number**
- **flip every bit**
- **then add one**

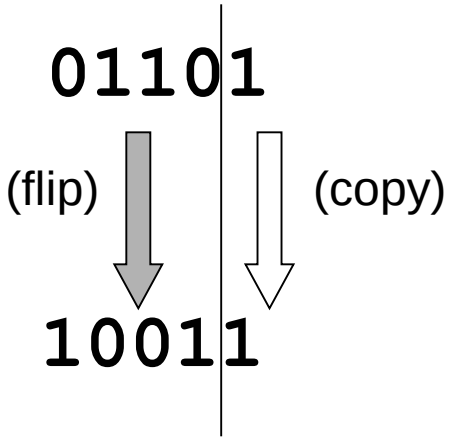


Two's Complement Shortcut

To find the negative of a number:

- **copy bits from right to left until (and including) the first "1"**
- **flip remaining bits to the left**

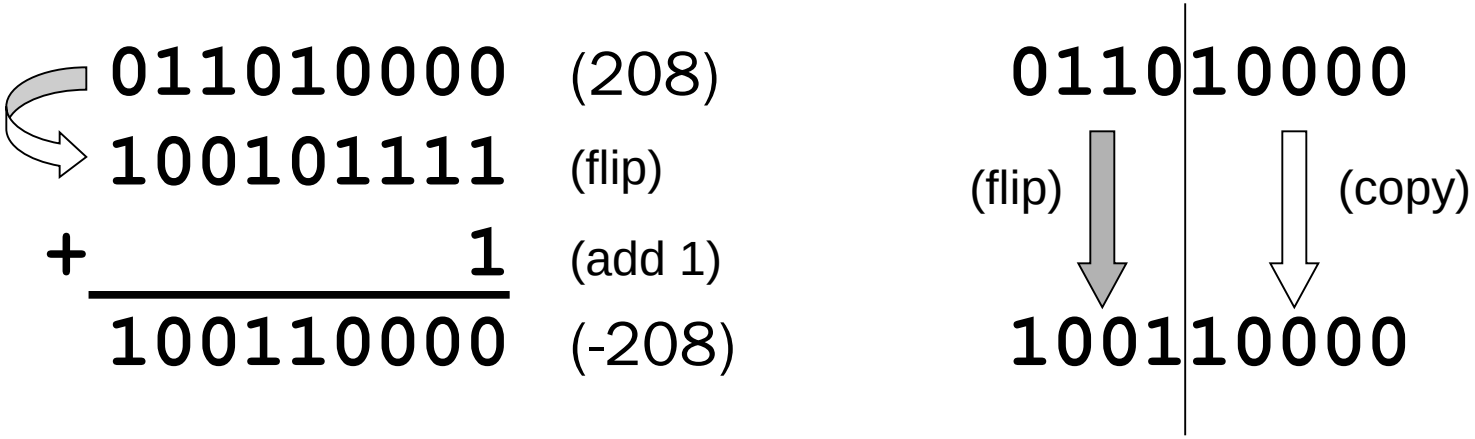




Two's Complement Shortcut

To find the negative of a number:

- **copy bits from right to left until (and including) the first "1"**
- **flip remaining bits to the left**



Two's Complement Signed Integers

MS bit is sign bit – it has weight -2^{n-1}.

Range of an n-bit number: -2^{n-1} through $2^{n-1} - 1$.

- **The most negative number (-2^{n-1}) has no positive counterpart.**

-2^3	2^2	2^1	2^0	
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7

-2^3	2^2	2^1	2^0	
1	0	0	0	-8
1	0	0	1	-7
1	0	1	0	-6
1	0	1	1	-5
1	1	0	0	-4
1	1	0	1	-3
1	1	1	0	-2
1	1	1	1	-1

Converting 2's Complement Number to Decimal

1. **If leading bit is one, take two's complement to get a positive number.**
2. **Add powers of 2 that have "1" in the corresponding bit positions.**
3. **If original number was negative, add a minus sign.**

$$X = 01101000_{two}$$
$$= 2^6+2^5+2^3 = 64+32+8$$
$$= 104_{ten}$$

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

Assuming 8-bit 2's complement numbers.

Converting 2's Complement Number to Decimal

1. **If leading bit is one, take two's complement to get a positive number.**
2. **Add powers of 2 that have "1" in the corresponding bit positions.**
3. **If original number was negative, add a minus sign.**

$$X = 00100111_{two}$$
$$= 2^5+2^2+2^1+2^0 = 32+4+2+1$$
$$= 39_{ten}$$

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

Assuming 8-bit 2's complement numbers.

Converting 2's Complement Number to Decimal

1. **If leading bit is one, take two's complement to get a positive number.**
2. **Add powers of 2 that have "1" in the corresponding bit positions.**
3. **If original number was negative, add a minus sign.**

$$X = 11100110_{two}$$
$$-X = 00011010$$
$$= 2^4+2^3+2^1 = 16+8+2$$
$$= 26_{ten}$$
$$X = -26_{ten}$$

Assuming 8-bit 2's complement numbers.

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

Converting Decimal to 2's Complement

First Method: *Division*

1. **Find magnitude of decimal number. (Always positive.)**
2. **Divide by two – remainder is least significant bit.**
3. **Keep dividing by two until answer is zero, writing remainders from right to left.**
4. **Append a zero as the MS bit; if original number was negative, take two's complement.**

$X = 104_{ten}$

104/2	=	52	r0	*bit 0*
52/2	=	26	r0	*bit 1*
26/2	=	13	r0	*bit 2*
13/2	=	6	r1	*bit 3*
6/2	=	3	r0	*bit 4*
3/2	=	1	r1	*bit 5*
1/2	=	0	r1	*bit 6*

$X = 01101000_{two}$

Converting Decimal to Binary (2's C)

Second Method: *Subtract Powers of Two*

1. **Find magnitude of decimal number.**
2. **Subtract largest power of two less than or equal to number.**
3. **Put a one in the corresponding bit position.**
4. **Keep subtracting until result is zero.**
5. **Append a zero as MS bit; if original was negative, take two's complement.**

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

$X = 104_{ten}$

104 - 64	=	40	*bit 6*
40 - 32	=	8	*bit 5*
8 - 8	=	0	*bit 3*

$X = 01101000_{two}$

Operations: Arithmetic and Logical

Recall:
a data type includes *representation* and *operations*.

We now have a good representation for signed integers, so let's look at some arithmetic operations:

- **Addition**
- **Subtraction**
- **Sign Extension**

We'll also look at overflow conditions for addition.

Multiplication, division, etc., can be built from these basic operations.

Logical operations are also useful:

- **AND**
- **OR**
- **NOT**

Sign Extension

To add two numbers, we must represent them with the same number of bits.

If we just pad with zeroes on the left:

4-bit		**8-bit**	
0100	(4)	00000100	(still 4)
1100	(-4)	00001100	(12, not -4)

Instead, replicate the MS bit -- the sign bit:

4-bit		**8-bit**	
0100	(4)	00000100	(still 4)
1100	(-4)	11111100	(still -4)

Addition

As we've discussed, 2's comp. addition is just binary addition.

- **assume all integers have the same number of bits**
- **ignore carry out**
- **for now, assume that sum fits in n-bit 2's comp. representation**

```
  01101000  (104)        11110110  (-10)
+ 11110000  (-16)     +  11110111  (-9)
----------             ----------
  01011000  (88)         11101101  (-19)
```

Assuming 8-bit 2's complement numbers.

Subtraction

Negate subtrahend (2nd no.) and add.

- **assume all integers have the same number of bits**
- **ignore carry out**
- **for now, assume that difference fits in n-bit 2's comp. representation**

```
  01101000  (104)        11110110  (-10)
- 00010000  (16)       - 11110111  (-9)
  --------              --------
  01101000  (104)       11110110  (-10)
+ 11110000  (-16)     + 00001001  (9)
  --------              --------
  01011000  (88)        11111111  (-1)
```

Assuming 8-bit 2's complement numbers.

Overflow

If operands are too big, then sum cannot be represented as an *n*-bit 2's complement number.

```
  01000  (8)          11000  (-8)
+ 01001  (9)        + 10111  (-9)
  -----              -----
  10001  (-15)         01111  (+15)
```

We have overflow if:

- **signs of both operands are the same, and**
- **sign of sum is different.**

Another test -- easy for hardware:

- **carry into MS bit does not equal carry out (will be clear after we study full-adder in Ch.3)**

Limitations of integer representations

Most numbers are not integer!

- Even with integers, there are two other considerations:

Range:

- The magnitude of the numbers we can represent is determined by how many bits we use:
  - e.g. with 32 bits the largest number we can represent is about +/- 2 billion, far too small for many purposes.

Precision:

- The exactness with which we can specify a number:
  - e.g. a 32 bit number gives us 31 bits of precision, or roughly 9 figure precision in decimal representation.

We need another data type!

Real numbers

Our decimal system handles non-integer *real* numbers by adding yet another symbol, i.e. the decimal point (.) to make a *fixed point* notation:

- **e.g. 3,456.78 = $3*10^3 + 4*10^2 + 5*10^1 + 6*10^0 + 7*10^{-1} + 8*10^{-2}$**

The *floating point*, or scientific, notation allows us to represent very large and very small numbers (integer or real), with as much or as little precision as needed:

- **Unit of electric charge e = $1.602176462 \times 10^{-19}$ Coul.**
- **Volume of universe = 1×10^{85} cm^3**
  - **the two components of these numbers are called**
    - **the fraction (1.602176462) and the exponent (-19)**
    - **the fraction (1) and the exponent (85)**

Real numbers in binary

We mimic the decimal floating point notation to create a "hybrid" binary floating point number:

- **We first use a "binary point" to separate whole numbers from fractional numbers to make a fixed point notation:**
  - **e.g. 00011001.110 = $1*2^4 + 1*2^3 + 1*2^0 + 1*2^{-1} + 1*2^{-2}$ => 25.75**
    ($2^{-1} = 0.5$ and $2^{-2} = 0.25$, etc.)

- **We then "float" the binary point:**
  - **00011001.110 = 1.1001110 * 2^4**
    fraction: 1.1001110, exponent: 4

- **Now we have to express this without the extra symbols (*, 2, .)**
  - **by convention, we divide the available bits into three fields:**
    sign, exponent, fraction

IEEE 754 Floating-Point Standard (32 bits)

Use equivalent of "scientific notation": F x 2^E

Need to represent F (*fraction*), E (*exponent*), and sign.



$$N = (-1)^S \times 1.\text{fraction} \times 2^{\text{exponent}-127},\ 1 \le \text{exponent} \le 254$$

$$N = (-1)^S \times 0.\text{fraction} \times 2^{-126},\ \text{exponent} = 0$$

Floating Point Example

Single-precision IEEE floating point number:



- **Sign is 1 – number is negative.**
- **Exponent field is 01111110 = 126 (decimal).**
- **Fraction is 0.100000000000… = 0.5 (decimal).**

Value = $-1.5 \times 2^{(126-127)} = -1.5 \times 2^{-1} = -0.75$.

Text: ASCII Characters

ASCII Code: Character to Binary

0	0011 0000	O	0100 1111	m	0110 1101
1	0011 0001	P	0101 0000	n	0110 1110
2	0011 0010	Q	0101 0001	o	0110 1111
3	0011 0011	R	0101 0010	p	0111 0000
4	0011 0100	S	0101 0011	q	0111 0001
5	0011 0101	T	0101 0100	r	0111 0010
6	0011 0110	U	0101 0101	s	0111 0011
7	0011 0111	V	0101 0110	t	0111 0100
8	0011 1000	W	0101 0111	u	0111 0101
9	0011 1001	X	0101 1000	v	0111 0110
A	0100 0001	Y	0101 1001	w	0111 0111
B	0100 0010	Z	0101 1010	x	0111 1000
C	0100 0011	a	0110 0001	y	0111 1001
D	0100 0100	b	0110 0010	z	0111 1010
E	0100 0101	c	0110 0011	.	0010 1110
F	0100 0110	d	0110 0100	,	0010 0111
G	0100 0111	e	0110 0101	:	0011 1010
H	0100 1000	f	0110 0110	;	0011 1011
I	0100 1001	g	0110 0111	?	0011 1111
J	0100 1010	h	0110 1000	!	0010 0001
K	0100 1011	I	0110 1001	'	0010 1100
L	0100 1100	j	0110 1010	"	0010 0010
M	0100 1101	k	0110 1011	(	0010 1000
N	0100 1110	l	0110 1100	)	0010 1001
				space	0010 0000

Hexadecimal Notation

Hexadecimal (base-16) numbers are widely used

by *computer system designers* and *programmers*,

as they provide a

human-friendly representation

of binary values.

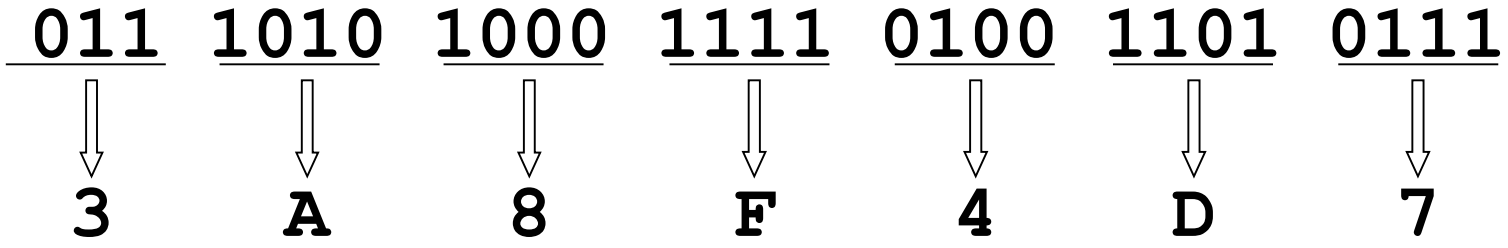
- **fewer digits – 1 digit per four bits**
- **less error prone**
  - **easy to corrupt long string of 1's and 0's**

Binary	Hex	Decimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Converting from Binary to Hexadecimal

Every four bits is a hexadecimal digit.

- **start grouping from right-hand side**



This is not a new machine representation, just a convenient way to write the number.

Text: ASCII Characters

ASCII: Maps 128 characters to 7-bit code.

- both printable and non-printable (ESC, DEL, …) characters

00	nul	10	dle	20	sp	30	0	40	@	50	P	60	`	70	p
01	soh	11	dc1	21	!	31	1	41	A	51	Q	61	a	71	q
02	stx	12	dc2	22	"	32	2	42	B	52	R	62	b	72	r
03	etx	13	dc3	23	#	33	3	43	C	53	S	63	c	73	s
04	eot	14	dc4	24	$	34	4	44	D	54	T	64	d	74	t
05	enq	15	nak	25	%	35	5	45	E	55	U	65	e	75	u
06	ack	16	syn	26	&	36	6	46	F	56	V	66	f	76	v
07	bel	17	etb	27	'	37	7	47	G	57	W	67	g	77	w
08	bs	18	can	28	(	38	8	48	H	58	X	68	h	78	x
09	ht	19	em	29	)	39	9	49	I	59	Y	69	i	79	y
0a	nl	1a	sub	2a	*	3a	:	4a	J	5a	Z	6a	j	7a	z
0b	vt	1b	esc	2b	+	3b	;	4b	K	5b	[	6b	k	7b	{
0c	np	1c	fs	2c	,	3c	<	4c	L	5c	\	6c	l	7c	|
0d	cr	1d	gs	2d	-	3d	=	4d	M	5d	]	6d	m	7d	}
0e	so	1e	rs	2e	.	3e	>	4e	N	5e	^	6e	n	7e	~
0f	si	1f	us	2f	/	3f	?	4f	O	5f	_	6f	o	7f	del

Interesting Properties of ASCII Code

What is relationship between a decimal digit ('0', '1', …) and its ASCII code?

What is the difference between an upper-case letter ('A', 'B', …) and its lower-case equivalent ('a', 'b', …)?

Given two ASCII characters, how do we tell which comes first in alphabetical order?

Are 128 characters enough?
(http://www.unicode.org/)

No new operations -- integer arithmetic and logic.

Logical Operations

Operations on logical TRUE or FALSE

- two states -- takes one bit to represent: TRUE=1, FALSE=0

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

A	NOT A
0	1
1	0

View *n*-bit number as a collection of *n* logical values

- operation applied to each bit independently

Examples of Logical Operations

AND

- useful for clearing bits
  - AND with zero = 0
  - AND with one = no change

OR

- useful for setting bits
  - OR with zero = no change
  - OR with one = 1

XOR

- Useful to see the difference
  - XOR same values = 0
  - XOR different values = 1

NOT

- unary operation -- one argument
- flips every bit

```
  11000101
AND 00001111
  --------
  00000101

  11000101
OR  00001111
  --------
  11001111

  11000101
XOR 00001111
  --------
  11001010

NOT 11000101
  --------
  00111010
```